

Will Richards - IMD 3002 - Prof. Chris Joslin
[Mandelbrot Embedding Tool - Part B \(Prototype\)](#)

Version	Date	Description of Completed Tasks
Overview - A	Feb. 15th	Created a broad over view and outline of the project
Prototype - B	Mar. 20th	Modified the earlier report and included the addition of: Validity checking for user selction, bounds checking for the set to be projected onto a surface, onion-skin projection, and Majority of GUI completed

Objectives

I fairly recently finished [The Black Swan](#) by [Nicholas Nassim Taleb](#) and have been inspired to create a Mandelbrot set representation in Maya. There are various levels of complexity in achieving an effect like this. I will do my best to complete all of them but there are of course **great lengths a project like this can be taken to**. time permitting, I'd like to:

0. Make extrude work on complex objects. (item #0 as this would not be considered to be in line with project definition but still complicated to achieve)
Extrude has not been implemented yet
1. Create a "mock" Mandelbrot generation on objects. Something that approximates a Mandelbrot set to a recognizable degree to ensure extrusion in either direction is possible
An onion-sketch does currently work and varies based on the z-value. Work needs to be done to further the embedding aspect, though extrusion appears as it should.
2. An "art" effect on the extruded surfaces. likely a RG gradient to stretch from either side to better fulfill the 3D art theme for this year
3. Begin experimentation with LoD in an "infinite" zoom / "infinite" Mandelbrot set generation based on params on Maya GUI.

To recap, the designed script should allow the user to embed / extrude a Mandelbrot fractal into the face(s) of an object and optionally create an effect on the surface that has been affected. The mandelbrot fractal will scale to a certain (not actually infinite) point depending on user's input in the window. It is difficult to quantify the "utility" of this application given the nature of 3D art, however I believe the repetitive nature of calculating any level of depth in a fractal lends itself well to Maya's API in scripting for 3D modelling. I also think this will be an interesting area to explore given the many different ways a fractal can be loaded into software.

Requirements / Specifications

To better outline how I will adhere to the aforementioned objectives, I am providing a more specific set of rules to follow in design this script:

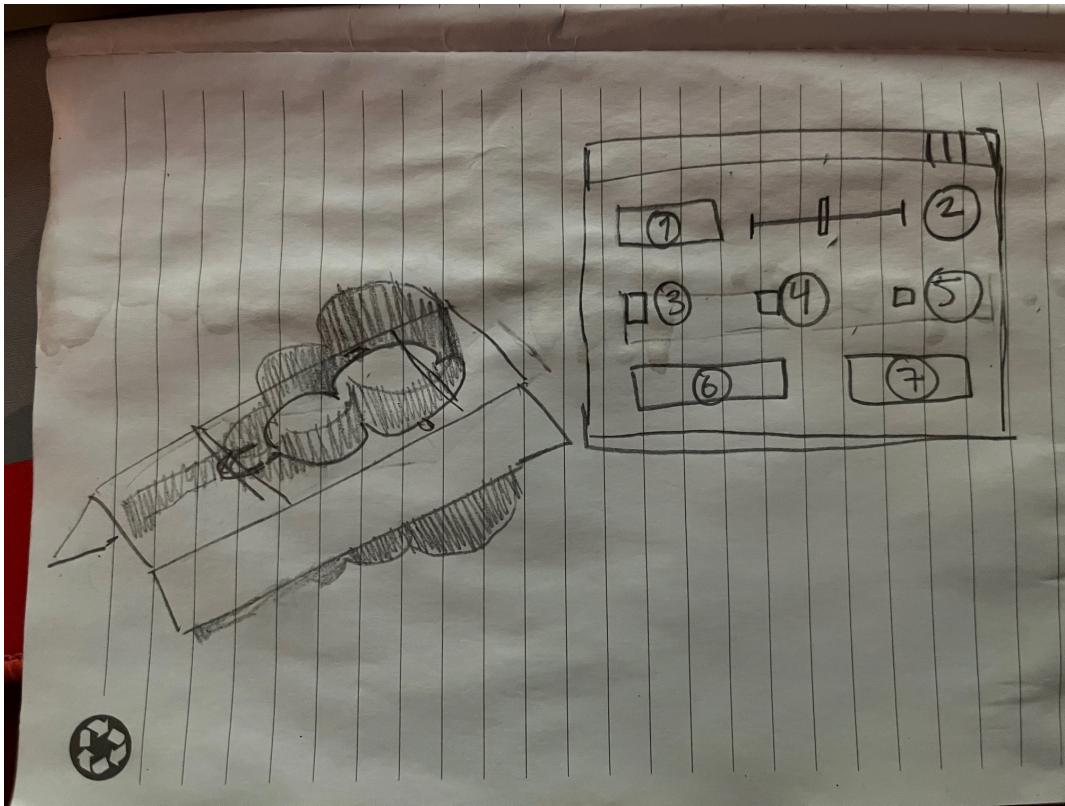
- Extrusion/Embedding in a mesh will be toggleable in the Gui along with an "onion-sketch layer" silhouette that displays how the effect will look.
- For complexity reasons. The user will only be able to extrude/embed along the face's normal. The z-value for this will be available to change from the GUI
- Extrusion should work on complex objects without destroying the mesh or creating non-manifold geometry. May need to clamp extrusions similar to how Blender's bevel tool works.
Currently weighing the plausibility of this sort of capability. Extrusion may pose to difficult to handle correctly and it may be more effective to create the object and have the user connect the geometry themselves
- Mandelbrot approximation needs to appear as such. The "fractal" created should appear like the real thing, minus the ability to scale infinitely, at least at first. The lowest LoD should be able to run no problem on low-end computers (or least computers capable of running Maya Haha).
- Given the specified LoD, the user should be able to scale to a point that appears correct on their mesh. Essentially LoD should be high enough fidelity that the user can utilize the script on a low-res mesh.
- The gradient effect will appear like a standard RG uv map and the two colours will be customizable from the GUI.
- While I have not experimented much with camera clipping I'd like to try implementing dynamic LoD loading as the user scrolls further in to view the mesh.
As a formal requirement for the project however, I'd like to ensure that LoD is statically decided upon by the user within the GUI

Coding Layout - Revised for Part B

Little changes have been made to the original plan, everything from helper math to onion layouts have been made. Physical embedding and extruding are the only parts to be done aside from colouring. These functions will take a large majority of the allotted time depending on the methods used to implement (proper topology / bool operations / simple object instantiation)

- `vector helpers` - A set of helpers to assist with the vector math to be done
- `validSelection()` - parses faces and runs error checks to ensure the selection is valid and can be used for any operation pertaining to the scope of this script
- `getBounds()` - runs to ensure that operations are confined to a valid area, operates on world space vertices and avoids duplicate counting of vertices in face parsing
- `mandelbrotIter()` - Generates the Mandelbrot set itself by iterating based on the number of iterations chosen by the user
- `renderPath()` - This function draws the Mandelbrot fractal outline along valid faces after being checked. `mandelBrotGrid()` is used to create the grid according to bounds set by `getBounds()`. This render path is what will be referenced to create the "onion-sketch" overlay and the final render. `renderPath()` will take inputs like width, height, LoD, and other modifiers
- `createOnion()` - defines a locator outline based on the vertices passed by the `renderPath()` function. `deleteOnion()` is also called to ensure the previous onion is deleted before creating a new one
- `extrudePath()` - Take an input of `renderPath()` vertices, subdivide the face to support the added vectors and extrude by the input z-value. Varying levels of cleanup may have to be done but the user depending on how far I get in this project. This function returns the faces after extrusion to be used for color changes
- `embedPath()` - Take an input of `renderPath()` vertices, subdivide the face to support the added vectors and embed by the input z-value. Varying levels of cleanup may have to be done but the user depending on how far I get in this project. This function returns the faces after embedding to be used for color changes
- `colorFractal` - Applies a customizable lambert material to color extrusions/embeddings based on a Mandelbrot-esque pattern

Design Sketches

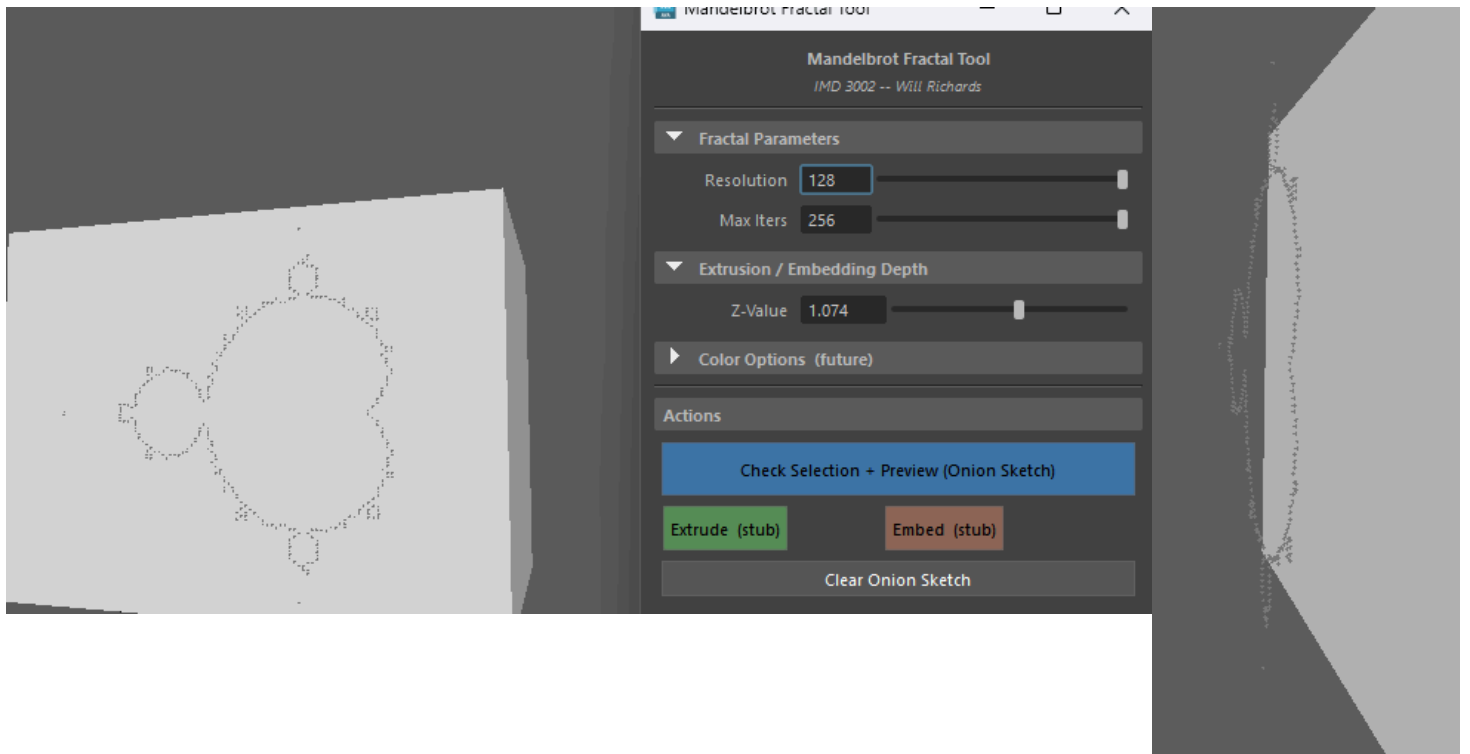


img GUI has ordered labels, explained below:

1. Check Selection - button to verify that the selected faces are valid before user attempts to perform action
 2. Z-Value Slider - This slider will affect the "onion-skin" overlay when user presses the "Check Selection" button. This is the value used for the extrusion and embedding
 3. Color? - button will toggle whether the user wants to color the shape after extrusion/embedding. This makes the color process more robust code-wise - off by default
 4. Col 1 - color picker for gradient - grey by default
 5. Col 2 - color picker for gradient - grey by default
 6. Extrude - button uses the params provided to extrude the fractal (up in this diagram)
 7. Embed - button uses the params provided to extrude the fractal (down in this diagram)
- *I may just combine extrude and embed into one button and allow negative z-value input**

The onion sketch makes things easier to comprehend for the user while posing in interesting challenge to implement, which will be fun. My hope is that it will be easy fast on the user side as it won't require geometry checking, though I will see after spending more time in Maya. This input scheme seems fairly intuitive and logical given the code needed to implement, though of course subject to change if I find a better way to lay things out.

Progress Recap Since Part A



A lot of progress has been made since the project overview, likley just over 45%. There is still much to do, but the following has been implemented:

- Bounds and validity checking - a lot of this has been simply parsing faces to vertices with lots of help from both maya and python documentation
- Mandelbrot - Pretty straightforward, Mandelbrot generation is a standard formula. The interesting part is creating a Mandelbrot grid using all the data generated by the bounds check. The received data is used to create a map to the bounds. This map / grid is then further used to place the Mandelbrot boundary lines within the shape's bounds. Boundary lines for the Mandelbrot set are created in essentially the same way an edge detection filter works, by checking current point against neighbour to detect a change.
- Onion Skin - the onion skin is just taking the vectors created by the mandelbrot set and placing locators at that point in world space. There is a callback within the GUI to allow for updates to onion skin but the updates are very slow and will likely be removed / simply not used
- Maya GUI - the last big thing introduced in this ppart of the project. The GUI is great as I learned a lot from Maya documentation regarding lambdas, callbacks, and mutable interfaces for GUI params / widgets. I have linked this resource below.

Steps for Part C

Steps for part A will require more research in to the best (most doable) method to actually construct the shapes, weither by attaching them or generting a shape for the user to play with. Additionally, unless the reader knows of any resources pointing to dynamic loading of vertices relative camera space / distance based LoD within the cam space inside of Maya, the "progressive mesh" idea will have to be shelved for now. Personal research has not shwon of any way to load vectors dynamically as the user scrolls into view of the mesh.

Resources:

- [Used for Python String formatting](#)
- [Methods of polygonal mesh calculations - used for centroid math and applications](#)
- [Used for Mandelbrot calculations](#)
- [Fantastic Resource for maya API and GUI specific pfunctions](#)
- [Maya Documentation](#)